

GAM Connection Setup — From Zero

2026-09-21 · @Syeda Nimra Fatima · branch `auth-by-named-credential`

This assumes nothing exists yet: no GCP service account, no certificate, no Salesforce metadata deployed. Do these in order.

0. Prerequisites

- A Salesforce org on Enterprise/Unlimited/Developer/Performance edition, API v56.0+, with standard API access (not Professional/Essentials without an API add-on).
- Access to a Google Cloud project (or ability to create one) and admin access to the target GAM network.
- `openssl`, `keytool` (ships with any JDK), and `node` installed locally.
- Salesforce CLI (`sf`) authenticated against the target org.

1. Create the Google service account

1. In Google Cloud Console, open **IAM & Admin** → **Service Accounts** → **Create Service Account**. Any name/ID is fine — it just needs to be identifiable later (e.g. `gam-sa`).
2. Open the new service account → **Keys** tab → **Add Key** → **Create new key** → **JSON**. This downloads `service-account-key.json` — treat it as a secret, it contains the private key. Do not commit it anywhere.
3. Note the `client_email` field inside that JSON (looks like `name@project-id.iam.gserviceaccount.com`) — you'll need it in step 4.

2. Add the service account as a GAM API user

1. In Google Ad Manager, go to **Admin** → **Global settings** and note the **Network code** — you'll need it later in Apex.
2. Go to **Admin** → **Access & authorization** → **Users** (or your GAM UI's equivalent) → add a new user using the service account's `client_email` as the identifier, and assign it a role with the API permissions you need (Administrator while validating; scope it down afterward).
3. Confirm the account shows as active before moving on — nothing downstream will work if this step is skipped.

3. Turn the JSON key into a Salesforce-importable certificate

Do this on your machine, not in Salesforce:

```
# 1. Pull the private key out of the JSON (it's stored JSON-escaped, with
literal \n's)
node -e "const k=require('./service-account-key.json');
require('fs').writeFileSync('sa-key.pem', k.private_key)"

# 2. Wrap it in a throwaway self-signed cert
openssl req -new -x509 -key sa-key.pem -out sa-cert.pem -days 3650 -subj
"/CN=gam-service-account"

# 3. Convert to PKCS12 — the -name value becomes the keystore alias
openssl pkcs12 -export -in sa-cert.pem -inkey sa-key.pem -out sa-key.p12 -
name gam_service_account_key -passout pass:changeit

# 4. Convert PKCS12 to JKS — Salesforce's "Import from Keystore" only accepts
JKS
keytool -importkeystore -srckeystore sa-key.p12 -srcstoretype PKCS12 -
srcstorepass changeit -destkeystore sa-key.jks -deststoretype JKS -
deststorepass changeit

# 5. Confirm the alias before importing
keytool -list -keystore sa-key.jks -storepass changeit
```

Two gotchas that will otherwise cost you an hour:

- The alias you pass to `-name` in step 3 becomes the certificate's Developer Name in Salesforce, and **it must not contain a hyphen** (`gam-sa` fails with `Keystore contains an entry whose alias is not acceptable for import`; `gam_service_account_key` works).
- The PKCS12 export silently **lowercases the alias** regardless of the case you typed. Don't assume the certificate's real name matches what you typed — verify it after import (step 4 below).

4. Import the certificate into Salesforce

1. Setup → **Certificate and Key Management** → **Import from Keystore** → select `sa-key.jks`, enter the password (`changeit` above).

2. If the page throws an unrelated error (a blank "Data Not Available" list page is a known one-off UI glitch), just navigate back to Certificate and Key Management fresh and retry.
3. Confirm the certificate's **actual** Developer Name: Setup → Certificate and Key Management → find it in the list, or run `sf org list metadata --metadata-type Certificate`. Use that exact (lowercase) name going forward — don't trust what you typed.
4. Delete the local `sa-key.pem`, `sa-cert.pem`, `sa-key.p12`, `sa-key.jks` — none need to survive past this step.

5. Deploy this repo's metadata

1. Clone this repo and check out `auth-by-named-credential`.
2. Open `force-app/core/main/default/externalCredentials/GAM_API_Cred.externalCredential-meta.xml` and:
 - Replace `REPLACE_WITH_SERVICE_ACCOUNT_CLIENT_EMAIL` (the `iss` claim) with the real `client_email` from step 1.
 - Set the `SigningCertificate` and `kid` parameter values to the certificate's real Developer Name from step 4.
3. Deploy everything: `sf project deploy start` (or dry-run first: `sf project deploy start --dry-run`). This pushes the External Credential, Named Credential, Permission Set, both Apex classes, and the `gamConnectionStatus` LWC.
4. In `force-app/core/main/default/classes/GAMNetworkServiceClient.cls`, confirm `NETWORK_CODE` matches your GAM network code from step 2, and that `GAM_API_VERSION` isn't a sunset GAM SOAP version (check developers.google.com/ad-manager/api/deprecation).

6. Grant access

1. Setup → Permission Sets → **GAM API Access** → **Manage Assignments** → assign to whichever user runs the integration (the integration user, or the running user of any scheduled/batch job that will call GAM).

7. Verify end to end

Either:

- Drop the `gamConnectionStatus` Lightning component onto any App or Home page (App Builder → drag it on) and open that page, or
- Run `scripts/apex/verify-token-caching.apex` in Execute Anonymous, or call `new GAMNetworkServiceClient().getCurrentNetwork()` directly.

A working result shows `Connected` with your real network code, display name, currency, and time zone. `AuthError` means the JWT/certificate/claims are wrong somewhere in steps 3–5; `ApiError` means auth succeeded but GAM rejected the SOAP call itself (usually a network code or API-user role problem from step 2).

Repeat steps 3–6 for every additional org this needs to run in — the certificate import can never be automated or deployed, it's manual per org, every time.